

Richtlinien für Swissdec-Übermittlungen: Sicherheit (Transmitter)

Version 2024.05

Die Richtlinien für Lohndatenübermittlung wurden in Zusammenarbeit mit folgenden Beteiligten erarbeitet:

- Suva
- eAHV / IV
- Konferenz der kantonalen Ausgleichskassen
- Vereinigung der Verbandsausgleichskassen
- Steuerverwaltung des Kantons Bern
- Schweizerische Steuerkonferenz
- Eidgenössische Steuerverwaltung
- Bundesamt für Statistik
- Schweizerischer Versicherungsverband

Herausgeber

Swissdec
Fluhmattstrasse 1
6004 Luzern

<http://www.swissdec.ch>

Versionsgeschichte

Version 20240626	26.06.2024	pfr
<i>Kapitel 4: KLEv1.1, SUAv1.1 und EPOv0.9 hinzugefügt</i>		
Version 20200220	11.07.2023	pfr
<i>Kapitel 2: Vorgaben gemäss OpenJDK 17 Tabelle 3.1. Empfohlene Algorithmen für den Bereich der Signatur</i>		
Version 20200220	26.10.2020	ab
<i>Ausgabe zur Detaillierung des Kapitels 4.3.2. Zertifikat erneuern</i>		
Version 20200220	23.04.2020	ifl
<i>Ausgabe für KLE, SUA und neu ELM V5</i>		
Version 20190301	16.07.2019	ab
<i>Ausgabe für KLE und SUA mit zweifacher Signatur (ERP- und UID- Zertifikate)</i>		
Version 20130514	07.07.2016	ab
<i>Letzte Ausgabe für ELM V4.0 und einfacher ERP-Signatur</i>		

Konventionen in diesem Dokument

Folgende Schriftarten werden in diesem Dokument verwendet:

Text	Dokumentation
Text	Code
<Text>	XML-Element
[Text]	Referenz auf ein anderes Dokument

Die Verbindlichkeit von Anforderungen ist wie folgt definiert:

Verbindlichkeit	Wort
Pflicht	<i>muss</i>
Wunsch	<i>soll (sollte)</i>
Absicht	<i>wird</i>
Vorschlag	<i>kann</i>

Tabelle 1. Verbindlichkeit von Anforderungen

Achtung

Für das konzeptionelle Verständnis genügen oft ältere Schemabilder, d.h. verbindlich sind immer nur die offiziellen XML-Files.

Inhaltsverzeichnis

- 1. Einführung 1
- 2. Sicherheit auf dem Transport Layer 2
- 3. Sicherheit auf dem Application Layer 3
 - 3.1. Fachliche Sicht 3
 - 3.2. Organisatorische Sicht 3
 - 3.3. Technische Sicht 3
- 4. Distributor und Swisssdec Standards 10
 - 4.1. Lohnstandard-CH (ELM) 10
 - 4.2. Leistungsstandard-CH (KLE) 12
 - 4.3. Swisssdec Unternehmens-Authentifizierung (SUA) 13
 - 4.4. Profilstandard-CH (EPO) 15
- 5. Skizzen zu den Zertifikaten 16
 - 5.1. Übersicht 16
 - 5.2. Übersicht mit SUA 17
 - 5.3. Application Layer 18
- A. Referenzen 19

Abbildungsverzeichnis

1.1. Schutzzielabdeckung	1
3.1. Reihenfolge Signatur und Verschlüsselung	4
3.2. zweifach Signatur	6
3.3. Request mit Signatur	7
3.4. Response mit Signatur Confirmation	8
5.1. Übersicht Zertifikate und ihre Verwendung	16
5.2. Übersicht Zertifikate und ihre Verwendung mit SUA	17
5.3. Übersicht Zertifikate und ihre Verwendung auf dem Application Layer mit SUA	18

Tabellenverzeichnis

1. Verbindlichkeit von Anforderungen	iv
3.1. Empfohlene Algorithmen für den Bereich der Signatur	5
3.2. Folgende Wahl der Algorithmen ist auf Transmitterseite mindestens Pflicht	9
4.1. ELM V4.0 Operation Protection Level	10
4.2. ELM V5.0 Operation Protection Level	11
4.3. KLE V1.0 Operation Protection Level	12
4.4. KLE V1.1 Operation Protection Level	12
4.5. SUA V1.0 Operation Protection Level	13
4.6. SUA V1.0 Renew Operation Protection Level	13
4.7. SUA V1.1 Operation Protection Level	14
4.8. SUA V1.1 Renew Operation Protection Level	14
4.9. EPO V0.9 Operation Protection Level	15
5.1. Bemerkungen zu den Swissdec Standards	16
5.2. Bemerkungen zu SUA	17
5.3. Bemerkungen zum Application Layer	18

1 Einführung

Potentiell werden pro Jahr bis zu 300'000 Unternehmen ihre Löhne, Ereignisse und weitere Daten via dem Distributor an die entsprechenden Stellen (z.B. Sozialversicherung oder Behörde) gemeldet. Da es sich um sensitive Daten handelt, sind entsprechende Vorkehrungen zu treffen, damit die Datensicherheit gewährleistet werden kann. Die Sicherheit ist also ein sehr wichtiger Aspekt in den gesamten Swissdec-Prozessen.

Bei der Datensicherheit wird oft zwischen **Transport**- und **Application**-Layer unterschieden. Diese Layer werden deshalb in den nachfolgenden Abschnitten getrennt behandelt.

Das Konzept mit der einfachen Signatur (bis zum Lohnstandard-CH (ELM) Version 4.0) wird neu (2019) um die SUA (Swissdec Unternehmens-Authentifizierung) erweitert. Im Leistungsstandard-CH (KLE) werden vermehrt Daten bidirektional ausgetauscht und aus Gründen des Datenschutzes muss die Authentifizierung des Unternehmens erhöht werden.

Aktuell gelten folgende Anforderungen:

Lohnstandard-CH (ELM) Version 5.0

- Der Transmitter des ERP-Systems *kann* UID Signatur SUA verwenden (optional).
- Der Endreceiver von Versicherer & Behörden *kann* SUA prüfen (optional).

Leistungsstandard-CH (KLE) Version 1.0 und Version 1.1

- Der Transmitter des ERP-Systems *muss* UID Signatur SUA verwenden.
- Der Endreceiver von Versicherer & Behörden *muss* SUA prüfen.

SUA (Swissdec Unternehmens-Authentifizierung)

■ Schutzzielabdeckung

**** Mit SUA wird in Zukunft zwischen dem ERP / Lohnbuchhaltung und dem Distributor die Authentizität und Verbindlichkeit erhöht.**

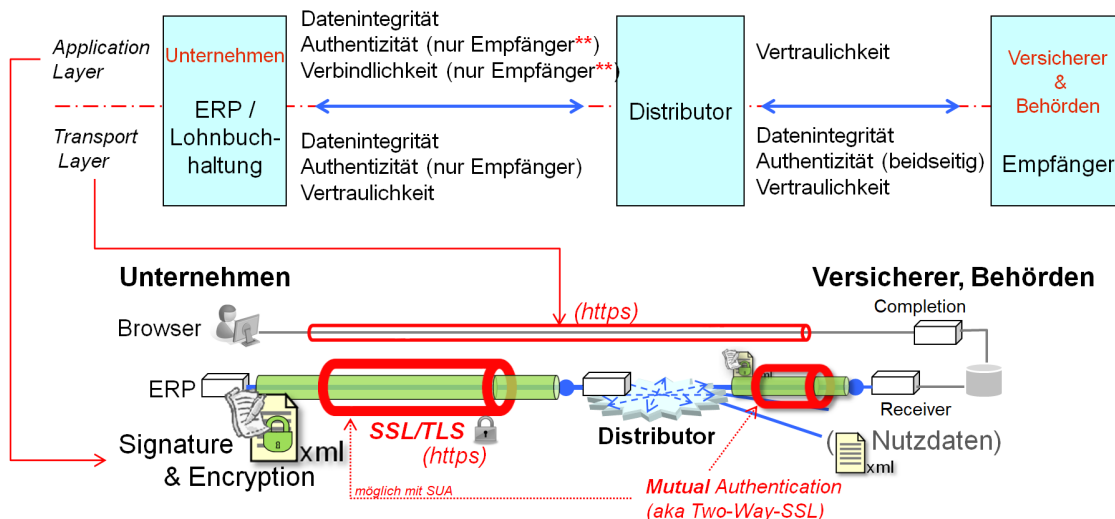


Abbildung 1.1. Schutzzielabdeckung

2 Sicherheit auf dem Transport Layer

Auf der Transportebene ist die Übermittlung per SSL/TLS (für Details siehe [RFC2246]) für alle beteiligten Systeme obligatorisch. Es kommen nur Sitzungsschlüssel zum Einsatz, die einen hohen Sicherheitsstandard erfüllen. Konkret bedeutet dies, dass die Sitzungsschlüssel eine Länge von mindestens 128bit aufweisen müssen, was die Auswahl der zu verwendenden Algorithmen einschränkt (IDEA, AES128: 128bit, 3DES: 168bit, AES256: 256bit). Der Distributor wird keine Verbindungen zulassen, die nicht diesen Anforderungen genügen.

Zurzeit gelten folgende Einschränkungen:

- Keine SSL Version ≤ 3 , kein TLSv1.0 und TLSv1.1
- Keine RC4, DES, MD5withRSA, 3DES_EDE_CBC, anon oder NULL Ciphers
- Keine DH keySize < 1024
- Keine EC keySize < 224

Anmerkung

Dies entspricht der Vorgaben des OpenJDK 17.0.2 (2022-01-22)/pfr

Mit dem Einsatz von SSL/TLS werden in Bezug auf die übertragenen Daten folgende Schutzziele erreicht:

- Vertraulichkeit (Daten werden verschlüsselt übermittelt)
- Authentizität (Empfänger, durch Server-Zertifikat)
- Datenintegrität (Manipulationen der Daten während der Übermittlung werden detektiert)

Es ist zu beachten, dass auf Transmitter-Seite bisher keine Client-Zertifikate zur Authentisierung zum Einsatz kommen. Diese Client-Zertifikate bezogen sich bisher nur auf den ERP-Hersteller und ein gegenseitigen Authentisierung (Mutual TLS) brachte dadurch wenig Nutzen.

Achtung

Mit der Einführung von SUA (Swissdec Unternehmens-Authentifizierung) *kann* sich nun auch der Client bzw. der Transmitter authentisieren.

3 Sicherheit auf dem Application Layer

Die Übermittlung der Daten per SSL/TLS bietet nur Punkt-zu-Punkt Sicherheit. Das heisst, die Daten sind nur während der Übermittlung geschützt. Sobald sie beim Empfänger ankommen, liegen diese wieder in Klartext vor und z.B. Veränderungen am Dokument sind nicht mehr erkennbar. An diesem Punkt setzt die Webservice Security auf XML-Ebene an. Durch die Verschlüsselung und Signierung der XML-Daten (Request und Response) kann die Datenintegrität auch nach der Übermittlung gewährleistet werden. Sie bietet also End-zu-End Sicherheit der Daten bezüglich Integrität, Authentizität und Verbindlichkeit. Für die Umsetzung werden ausschliesslich etablierte Standards eingesetzt. Konkret basiert die Digitale Signatur, wie sie im Lohnstandard-CH verwendet wird, auf dem Web Services Security Standard 1.0 von Oasis [WSS] und dem dazugehörigen X.509 Token Profile [WSSX509].

3.1. Fachliche Sicht

Teil des Swissdec Standards ist die Zertifizierung eines ERP-Systems. Nach erfolgreicher fachlicher und technischer Zertifizierung wird dem ERP-Hersteller ein digitales Zertifikat ausgestellt. Mit diesem wird durch die Signierung eines jeden SOAP-Requests die Zertifizierung nachgewiesen. Es ist zu beachten, dass das digitale Zertifikat an ein zertifiziertes ERP-System und nicht an ein Unternehmen gebunden ist. Die Schnittstelle ist jedoch so ausgelegt, dass zusätzlich zur Signatur des ERP-Systems auch weitere Signaturen möglich sind (z.B. Unternehmenssignatur, qualifizierte Signatur des verantwortlichen Lohnbuchhalters). Die dazu nötige Public-Key Infrastruktur ist in der Schweiz jedoch erst im Aufbau. (siehe SUA)

3.2. Organisatorische Sicht

Swissdec übernimmt die Rolle einer Certification Authority (CA) und Registration Authority (RA) für die Zertifikate der ERP-Hersteller. Sie stellt die Zertifikate aus und verwaltet sie.

Die Unternehmenszertifikate werden im Swissdec Standard durch SUA geregelt.

3.3. Technische Sicht

Die digitale Unterschrift, wie sie bei Swissdec verwendet wird, basiert auf dem OASIS Standard Web Services Security Standard Version 1.0 [WSS] und dem dazugehörigen X.509 Token Profile [WSSX509].

Aus technischer Sicht werden der ganze SOAP-Body und Teile des Headers signiert und anschliessend verschlüsselt. Swissdec legt dabei zusätzlich folgende Randbedingungen fest, die es zu berücksichtigen gilt.

2 Signatures and Encryption

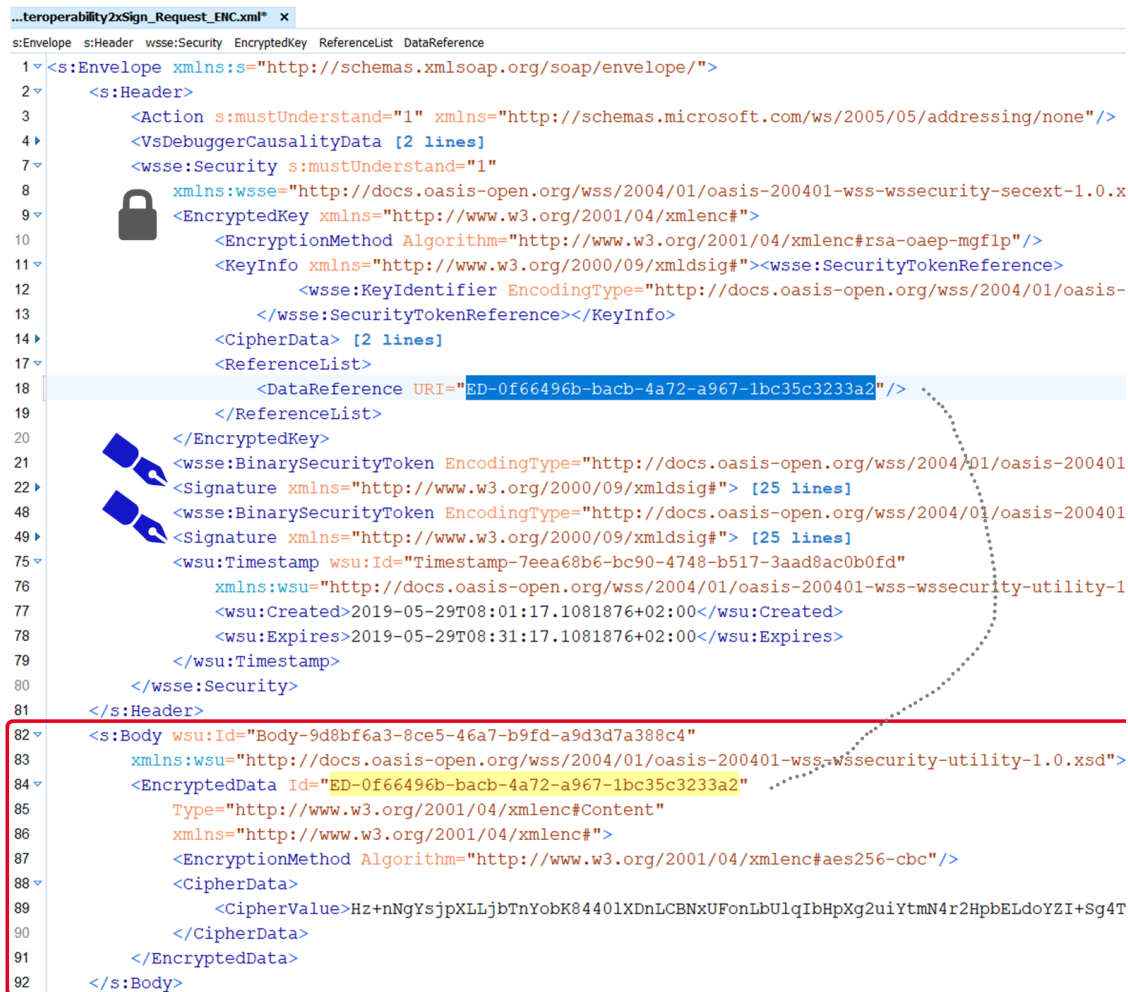


Abbildung 3.1. Reihenfolge Signatur und Verschlüsselung

3.3.1. Einfache oder zweifache Signatur

Der folgende Ablauf beschreibt die einfache und zweifache Signatur:

1. Es wird ausschliesslich mit X.509v3 Zertifikaten gearbeitet.
2. Mindestens das SOAP-Body Element und der Timestamp (siehe Punkt 4) *müssen* signiert werden. Zusätzlich signierte Elemente sind zulässig, müssen aber durch den Distributor verifiziert werden können. Das heisst, es dürfen nur Zertifikate von offiziellen Certificate Authorities zum Einsatz kommen.
3. Das Zertifikat *muss* immer in Form eines Base64-codierten Binary Security Tokens (Element <BinarySecurityToken>), welches mittels einer direkten Referenz (Element <SecurityTokenReference>) referenziert wird, mitgeschickt werden.
4. Um Replay-Attacken erkennen zu können, *muss* die Meldung ein <Timestamp> Element besitzen. Ein Expires-Element ist immer zu definieren. Die Länge der Gültigkeit wird vom Transmitter bestimmt. Das Timestamp-Element muss signiert werden.
5. Das mustUnderstand-Flag auf dem Security-Element muss auf 1 bzw. true gesetzt werden.

6. Die Requests aller Operationen (CheckInteroperability, DeclareSalary, usw.) ausser **Ping** müssen 1x oder 2x signiert werden.

- 1x signiert: Die Signatur mit dem ERP-Zertifikat erstellen.
- 2x signiert: Die erste Signatur mit dem ERP-Zertifikat erstellen und anschliessend die zweite Signatur mit dem UID-Zertifikat erstellen.

Die Anforderungen für 1x oder 2x Signieren finden sich bei den entsprechenden Standards und ihren Protection Levels (siehe Kapitel 4, *Distributor und Swissdec Standards*).

7. Der Transmitter muss in der Lage sein, die signierte Antwort des Receivers zu prüfen. Antworten mit einer ungültigen Signatur dürfen nicht akzeptiert werden.

Dabei wird dann auch die Signature Confirmation geprüft, d.h. der Distributor bestätigt den Erhalt der Nachricht, indem die Request-Signatur im Response gespiegelt und signiert wird. (siehe Abschnitt 3.3.1.1, „Signature Confirmation“)

8. Folgende Wahl der Algorithmen wird empfohlen:

Verwendung	Spezifikation
Canonicalization	http://www.w3.org/2001/10/xml-exc-c14n#
Signature	http://www.w3.org/2001/04/xmldsig-more#rsa-sha256
Digest	http://www.w3.org/2001/04/xmlenc#sha256

Tabelle 3.1. Empfohlene Algorithmen für den Bereich der Signatur

In folgendem Bild wird ein XML Instanzbeispiel mit zweifacher Signatur gezeigt. Es wird je mit dem ERP-Zertifikat und dem UID-BFS Zertifikat signiert. Beide Signaturen signieren immer Body und Timestamp.

2 Signatures

```
1 <s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
2   <s:Header>
3     <Action s:mustUnderstand="1" xmlns="http://schemas.microsoft.com/ws/2005/05/addressing"
4     <VsDebuggerCausalityData [2 lines]
7     <wsse:Security
8       xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-sec
9       <wsse:BinarySecurityToken EncodingType="http://docs.oasis-open.org/wss/2004/01/oa
10      <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
11        <SignedInfo>
12          <CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#
13          <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"/>
14          <Reference URI="#Timestamp-7eea68b6-bc90-4748-b517-3aad8ac0b0fd">
15            <Transforms> [2 lines]
16            <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
17            <DigestValue>VOl1VVAF+txrTCIldF2TtLmAFm4=</DigestValue>
18          </Reference>
19          <Reference URI="#Body-9d8bf6a3-8ce5-46a7-b9fd-a9d3d7a388c4"> [6 lines]
20        </SignedInfo>
21        <SignatureValue>R4h2LR/YyjQiucVy6BZLIeDRKlp3HvayBIL85xUn69T1T991R4/6APAF+P25oI
22        <KeyInfo><wsse:SecurityTokenReference>
23          <wsse:Reference URI="#BST-da55e2c1-c9bc-42f1-a252-5c82d95a8dea"
24            ValueType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss
25            <!-- DebugComment: Subject is [ OID.2.5.4.97=NTRCH-CHE-108.603.102, CN=
26          </wsse:SecurityTokenReference></KeyInfo>
27        </Signature>
28        <wsse:BinarySecurityToken EncodingType="http://docs.oasis-open.org/wss/2004/01/oa
29        <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
30          <SignedInfo>
31            <CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#
32            <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"/>
33            <Reference URI="#Timestamp-7eea68b6-bc90-4748-b517-3aad8ac0b0fd"> [6 lines]
34            <Reference URI="#Body-9d8bf6a3-8ce5-46a7-b9fd-a9d3d7a388c4">
35              <Transforms> [2 lines]
36              <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
37              <DigestValue>1BiOznle77fyMtN9/Owd6VWpn3I=</DigestValue>
38            </Reference>
39          </SignedInfo>
40          <SignatureValue>RpJ0L6i/rhpFK+dZaHBn5rxmACuo/4TQxiBH9IwW3Xw99qqsf4YC+ubKsgcqf
41          <KeyInfo><wsse:SecurityTokenReference>
42            <wsse:Reference URI="#BST-215677ea-4f59-43c3-9eaf-bde00e2f4684"
43              ValueType="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss
44              <!-- DebugComment: Subject is [ OID.0.9.2342.19200300.100.1.1=4.0, CN=
45            </wsse:SecurityTokenReference></KeyInfo>
46          </Signature>
47          <wsu:Timestamp wsu:Id="Timestamp-7eea68b6-bc90-4748-b517-3aad8ac0b0fd"
48            xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-
49            <wsu:Created>2019-05-29T08:01:17.1081876+02:00</wsu:Created>
50            <wsu:Expires>2019-05-29T08:31:17.1081876+02:00</wsu:Expires>
51          </wsu:Timestamp>
52        </wsse:Security>
53      </s:Header>
54      <s:Body wsu:Id="Body-9d8bf6a3-8ce5-46a7-b9fd-a9d3d7a388c4"
55        xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-
56        <CheckInteroperability
57          xmlns="http://www.swissdec.ch/schema/oa/20190301/OrganizationAuthenticationService
58          <UserAgent xmlns="http://www.swissdec.ch/schema/common/20180901/Common">
59            <Producer>itServe AG</Producer>
60            <Name>tester</Name>
```

Abbildung 3.2. zweifach Signatur

In den beiden nächsten Bildern wird der Zusammenhang zwischen der Signatur und der Signature Confirmation aufgezeigt.

3.3.1.1. Signature Confirmation

Als erster Teil wird ein Request mit seiner Signatur <SignatureValue> gezeigt:

[1] siehe Signature Confirmation (Line 29)

```
2 <s:Header>
3   <Action s:mustUnderstand="1" xmlns="http://schemas.microsoft.com/ws/2005/05/addressing/n
4   <VsDebuggerCausalityData [2 lines]
5   <wsse:Security
6     xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secex
7     <wsse:BinarySecurityToken EncodingType="http://docs.oasis-open.org/wss/2004/01/oasis
8     <Signature xmlns="http://www.w3.org/2000/09/xmldsig#"
9       <SignedInfo>
10        <CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#"
11        <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa-sha1"/>
12        <Reference URI="#Timestamp-7eea68b6-bc90-4748-b517-3aad8ac0b0fd">
13          <Transforms> [2 lines]
14          <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1"/>
15          <DigestValue>VOi1VVAf+txrTCiIdF2TtLmAFm4=
```

Abbildung 3.3. Request mit Signatur

Nun kommt der Response-Teil mit der entsprechenden <SignatureConfirmation> dazu:

```
1 <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
2   <soap:Header>
3     <wsse:Security soap:mustUnderstand="1"
4       xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecuri
5       xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurit
6       <wsse:BinarySecurityToken EncodingType="http://docs.oasis-open.org/wss/2004/
7       <ds:Signature Id="SIG-00b70b09-8426-4194-b028-4fa9c35349d2"
8         xmlns:ds="http://www.w3.org/2000/09/xmldsig#">
9         <ds:SignedInfo>
10           <ds:CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-
14           <ds:SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#rsa
15           <ds:Reference URI="#id-011659ea-f683-4397-aeab-808584eddfdf"> [6 line
22           <ds:Reference URI="#TS-a4e67369-0c0e-4e69-85d2-2f8670f74a04"> [9 line
32           <ds:Reference URI="#SC-6039f617-a6e4-44e0-99d1-91dd0c6b858f"> [9 line
42           <ds:Reference URI="#SC-b554dbec-d847-41b9-a124-9121ba5cc3d9">
43             <ds:Transforms> [5 lines]
49             <ds:DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sh
50             <ds:DigestValue>iPKX571zQv5X7ditLzDMEuwQY0Y=</ds:DigestValue>
51           </ds:Reference>
52         </ds:SignedInfo>
53         <ds:SignatureValue>nPCeG6sZssp6jBOBJm01Ae9GBLQWkhEA3tUjw09psP5YIuKD8F2md
54         <ds:KeyInfo Id="KI-c5c8fe2a-44ce-4853-9b01-2e5139c0e28d" [6 lines]
61       </ds:Signature>
62       <wsu:Timestamp wsu:Id="TS-a4e67369-0c0e-4e69-85d2-2f8670f74a04">
63         <wsu:Created>2019-05-29T06:01:17.243Z</wsu:Created>
64         <wsu:Expires>2019-05-29T06:16:17.243Z</wsu:Expires>
65       </wsu:Timestamp>
66       <wsse11:SignatureConfirmation [1] siehe Signature Confirmation
67         Value="R4h2LR/YyJQiucVy6BZLIeDRKlp3HvayBIL85xUn69T1T991R4/6APAF+P25oMaTd
68         wsu:Id="SC-b554dbec-d847-41b9-a124-9121ba5cc3d9"
69         xmlns:wsse11="http://docs.oasis-open.org/wss/oasis-wss-wssecurity-secext
70       <wsse11:SignatureConfirmation
71         Value="RpJ0L6i/rhpFK+dZaHBn5rxmACuo/4TQxiBH9IwW3Xw99qqsF4YC+ubKsgcqfviUe
72         wsu:Id="SC-6039f617-a6e4-44e0-99d1-91dd0c6b858f"
73         xmlns:wsse11="http://docs.oasis-open.org/wss/oasis-wss-wssecurity-secext
74       </wsse:Security>
75     </soap:Header>
76     <soap:Body wsu:Id="id-011659ea-f683-4397-aeab-808584eddfdf"
77       xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-ut
78     <ns4:CheckInteroperabilityResponse
79       xmlns="http://www.swissdec.ch/schema/common/20180901/Common"
80       xmlns:ns2="http://www.swissdec.ch/schema/oa/20190301/OrganizationAuthenticat
81       xmlns:ns3="http://www.swissdec.ch/schema/oa/20190301/OrganizationAuthenticat
82       xmlns:ns4="http://www.swissdec.ch/schema/oa/20190301/OrganizationAuthenticat
83     <UserAgent>
84       <Producer>userAgentType.producer</Producer>
85       <Name>userAgentType.name</Name>
```

Abbildung 3.4. Response mit Signatur Confirmation

3.3.2. Verschlüsselung

Der WS-Security Standard [WSS] definiert auch die Verschlüsselung von SOAP-Daten. Swissdec verlangt die Verschlüsselung sämtlicher Requests (mit Ausnahme der Ping Operation) nach deren Signierung. Es müssen folgende Regeln eingehalten werden:

1. Es wird ausschliesslich mit X.509v3 Zertifikaten gearbeitet.
2. Reihenfolge: Signierung *muss* vor Verschlüsselung sein (Grund: Man will wissen, was man unterschreibt)
3. Alle Operationen ausser Ping *müssen* verschlüsselt werden.
4. Der ganze SOAP-Body *muss* verschlüsselt werden (Content):

{Content}{http://schemas.xmlsoap.org/soap/envelope/}Body

5. Folgende Wahl der Algorithmen wird empfohlen:

Verwendung	Spezifikation
EncryptedKey/EncryptionMethod	http://www.w3.org/2001/04/xmlenc#rsa-oaep-mgf1p
EncryptedData/EncryptionMethod	http://www.w3.org/2001/04/xmlenc#aes256-cbc

Tabelle 3.2. Folgende Wahl der Algorithmen ist auf Transmitterseite mindestens Pflicht

Die zu verwendenden Algorithmen können sich jederzeit aus Sicherheitsgründen ändern. Dies wird jeweils von Swisssdec kommuniziert. Der Distributor wird nicht zwingend mit obengenannten Algorithmen verschlüsseln, weshalb der Transmitter so konfiguriert sein muss, dass er unterschiedliche Algorithmen akzeptiert.

4 Distributor und Swisdec Standards

Der Distributor überprüft und stellt sicher, dass die obgenannten Randbedingungen für Transmitter eingehalten werden, die Signatur gültig ist und korrekt verschlüsselt wird. Es bleibt die Frage, wie die Antworten des Distributors, abhängig vom Protection Level, aussehen. Dies soll nachfolgend geklärt werden.

Anmerkung

Werden auf dem Referenz-Distributor Probleme mit der Signatur oder Verschlüsselung festgestellt, werden diese in der Antwort in Form von Notifications zurück gemeldet. Beachten Sie jedoch, dass in solchen Fällen in der Produktion aus Sicherheitsgründen keine detaillierten Fehlerbeschreibungen zu erwarten sind. Genauere Informationen zu möglichen Rückmeldungen sind im Notifications-Dokument [ACKNOTIF] zu finden.

4.1. Lohnstandard-CH (ELM)

4.1.1. ELMv4.0

TargetNamespace=

<http://www.swissdec.ch/schema/sd/20130514/SalaryDeclarationService>

Gemäss dem SalaryDeclarationService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or SD-Fault
Ping	- -	- -	SOAP-Fault
CheckInteroperability	1x Signature (ERP) 1x optionale Signature (UID) +Encryption	1x Signature (Distributor) +Encryption	SOAP-Fault
DeclareSalary und GetStatusFromDeclareSalary und GetResultFromDeclareSalary und SynchronizeContract und GetStatusFromSynchronizeContract und GetResultFromSynchronizeContract	1x Signature (ERP) 1x optionale Signature (UID) +Encryption	1x Signature (Distributor) +Encryption	SOAP-Fault SD-Fault: Signature +Encryption

Tabelle 4.1. ELM V4.0 Operation Protection Level

4.1.2. ELMv5.0

TargetNamespace=

<http://www.swissdec.ch/schema/sd/20200220/SalaryDeclarationService>

Gemäss dem SalaryDeclarationService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or SD-Fault
Ping	- -	- -	SOAP-Fault
CheckInteroperability	1x Signature (ERP) 1x optional Signature (UID) +Encryption	1x Signature (Distributor) +Encryption	SOAP-Fault
DeclareSalary und GetStatusFromDeclareSalary und GetResultFromDeclareSalary und DeclareMonthTotal und GetStatusFromDeclareMonthTotal und GetResultFromDeclareMonthTotal und GetDialog und ReplyDialog	1x Signature (ERP) 1x optionale Signature (UID) +Encryption +Check Response Signature Confirmation	1x Signature (Distributor) +Encryption +Set Response Signature Confirmation	SOAP-Fault SD-Fault: Signature +Encryption

Tabelle 4.2. ELM V5.0 Operation Protection Level

4.2. Leistungsstandard-CH (KLE)

4.2.1. KLEv1.0

targetNamespace=

<http://www.swissdec.ch/schema/id/20180901/IncidentDeclarationService>

Gemäss dem IncidentDeclarationService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or ID-Fault
Ping	--	--	SOAP-Fault
CheckInteroperability	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault
DeclareIncident und SynchronizeIncident	2x Signature (ERP, UID) +Encryption +Check Response Signature Confirmation	Signature +Encryption +Set Response Signature Confirmation	SOAP-Fault ID-Fault: Signature +Encryption

Tabelle 4.3. KLE V1.0 Operation Protection Level

4.2.2. KLEv1.1

targetNamespace=

<http://www.swissdec.ch/schema/id/20230901/IncidentDeclarationService>

Gemäss dem IncidentDeclarationService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or ID-Fault
Ping	--	--	SOAP-Fault
CheckInteroperability	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault
DeclareIncident und SynchronizeIncident	2x Signature (ERP, UID) +Encryption +Check Response Signature Confirmation	Signature +Encryption +Set Response Signature Confirmation	SOAP-Fault ID-Fault: Signature +Encryption

Tabelle 4.4. KLE V1.1 Operation Protection Level

4.3. Swissdec Unternehmens-Authentifizierung (SUA)

4.3.1. SUAv1.0 Registrierung und UID-BFS Zertifikat

targetNamespace=
<http://www.swissdec.ch/schema/oa/20190301/OrganizationAuthenticationService>

Gemäss dem OrganizationAuthenticationService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or OA-Fault
Ping	--	--	SOAP-Fault
CheckInteroperability	1x Signature (ERP) +Encryption	Signature +Encryption	SOAP-Fault
RegisterOrganization und GetResultFromRegisterOrganization und SignCertificate	1x Signature (ERP) +Encryption	Signature +Encryption	SOAP-Fault OA-Fault: Signature +Encryption

Tabelle 4.5. SUA V1.0 Operation Protection Level

4.3.2. SUAv1.0 Zertifikat erneuern

targetNamespace=
<http://www.swissdec.ch/schema/oa/20190301/OrganizationAuthenticationRenewService>

Gemäss dem OrganizationAuthenticationRenewService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or OA-Fault
CheckInteroperability	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault
RenewCertificate	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault OA-Fault: Signature +Encryption

Tabelle 4.6. SUA V1.0 Renew Operation Protection Level

4.3.3. SUAv1.1 Registrierung und UID-BFS Zertifikat

targetNamespace=
<http://www.swissdec.ch/schema/oa/20230301/OrganizationAuthenticationService>

Gemäss dem OrganizationAuthenticationService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or OA-Fault
Ping	--	--	SOAP-Fault
CheckInteroperability	1x Signature (ERP) +Encryption	Signature +Encryption	SOAP-Fault
RegisterOrganization und GetResultFromRegisterOrganization und SignCertificate	1x Signature (ERP) +Encryption	Signature +Encryption	SOAP-Fault OA-Fault: Signature +Encryption

Tabelle 4.7. SUA V1.1 Operation Protection Level

4.3.4. SUAv1.1 Zertifikat erneuern

targetNamespace=

<http://www.swissdec.ch/schema/oa/20230301/OrganizationAuthenticationRenewService>

Gemäss dem `OrganizationAuthenticationRenewService.wsdl` werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or OA-Fault
CheckInteroperability	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault
RenewCertificate	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault OA-Fault: Signature +Encryption

Tabelle 4.8. SUA V1.1 Renew Operation Protection Level

4.4. Profilstandard-CH (EPO)

4.4.1. EPOv0.9 (Pilot)

targetNamespace=

<http://www.swissdec.ch/schema/po/20230801/ProfileOrderService>

Gemäss dem ProfileOrderService.wsdl werden die Operationen mit den entsprechenden Protection Levels definiert.

Operation	Protection Level Request	Protection Level Response	SOAP-Fault or PO-Fault
Ping	--	--	SOAP-Fault
CheckInteroperability	2x Signature (ERP, UID) +Encryption	Signature +Encryption	SOAP-Fault
OrderProfile und GetStatusFromOrderProfile und GetResultFromOrderProfile	2x Signature (ERP, UID) +Encryption +Check Response Signature Confirmation	Signature +Encryption +Set Response Signature Confirmation	SOAP-Fault PO-Fault: Signature +Encryption

Tabelle 4.9. EPO V0.9 Operation Protection Level

5 Skizzen zu den Zertifikaten

Die folgenden Skizzen sollen die Verwendung der verschiedenen Zertifikate und privaten Schlüssel etwas verständlicher darstellen.

5.1. Übersicht

Diese Skizze zeigt das Zusammenwirken der unterschiedlichen Zertifikate über die Transport- und Application-Layer gemäss Swisdec Standards.

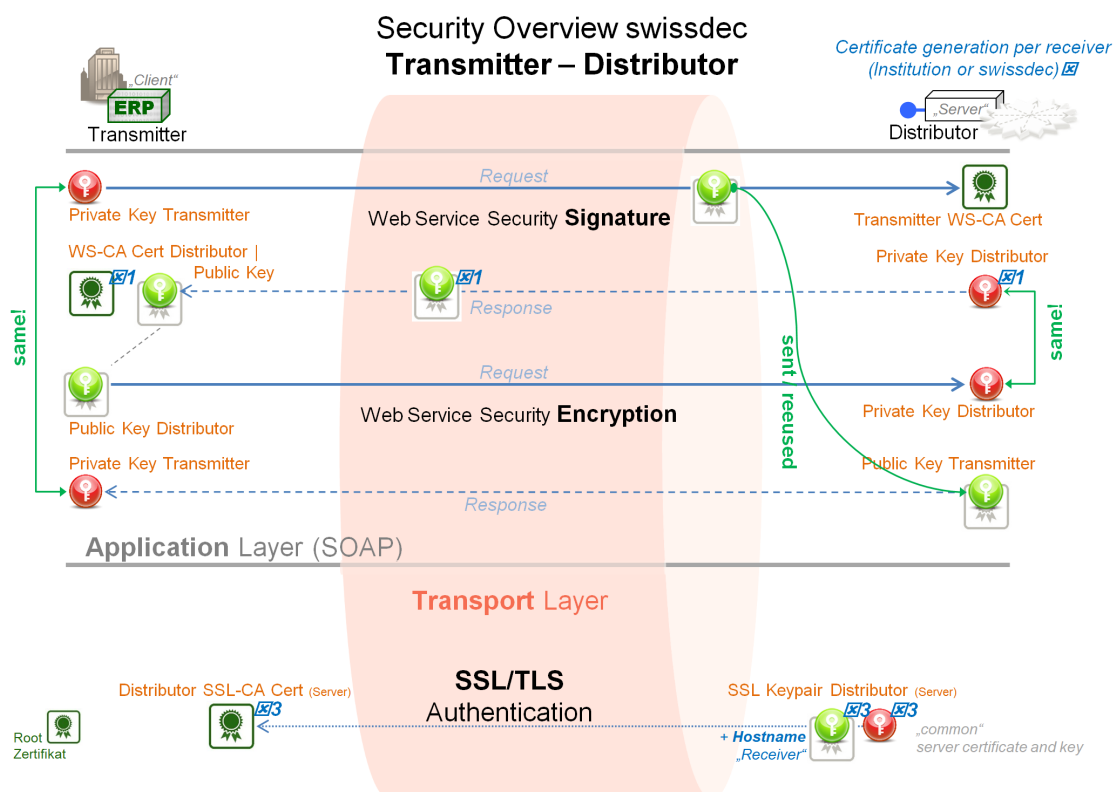


Abbildung 5.1. Übersicht Zertifikate und ihre Verwendung

	Privater Schlüssel (Private Key)
	Zertifikat mit öffentlichem Schlüssel (Public Key). Im Folgenden wird "öffentlicher Schlüssel" (Public Key) gleichgesetzt mit dem Zertifikat, welches den öffentlichen Schlüssel beinhaltet. In der Skizze finden sich folgende Zertifikate: • Application Layer mit ERP/Transmitter-Zertifikat (WS) und Distributor-Zertifikat (WS) • Transport Layer mit SSL/TLS Distributor-Zertifikat
	CA Zertifikat

Tabelle 5.1. Bemerkungen zu den Swisdec Standards

5.3. Application Layer

Diese Skizze zeigt das Zusammenwirken der unterschiedlichen Zertifikate nur auf dem Application- Layer.

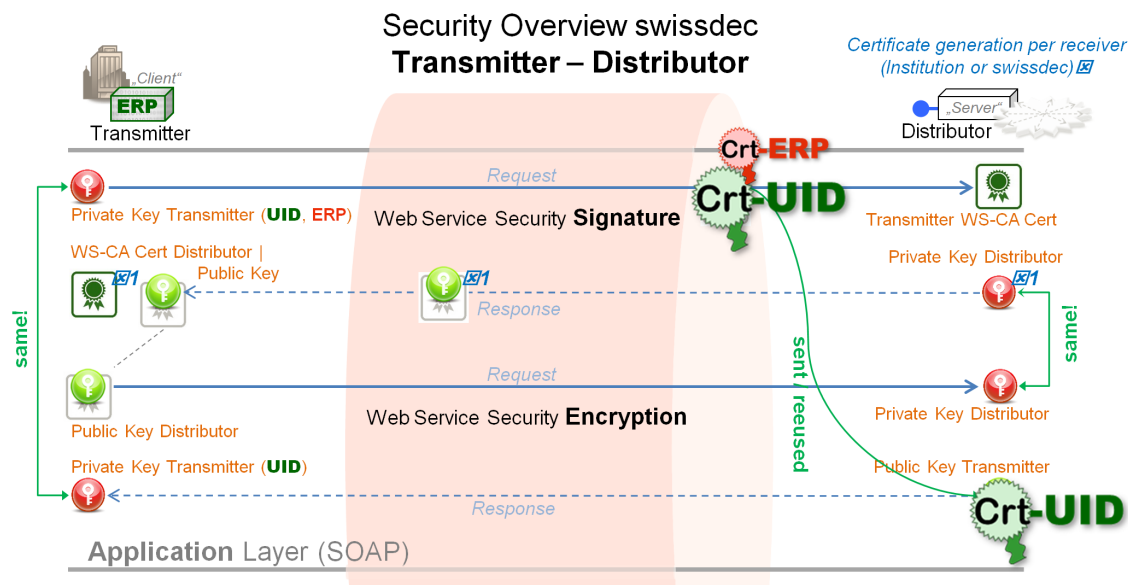


Abbildung 5.3. Übersicht Zertifikate und ihre Verwendung auf dem Application Layer mit SUA

Die Skizzen und die folgenden Bemerkungen zeigen die Verwendung von öffentlichen Schlüsseln, die in Zertifikaten liegen und den entsprechenden privaten Schlüsseln (Asymmetrisches Kryptosystem). Allerdings sind die Erklärungen einfach und schematisch. Diese sollen nur zeigen, auf welcher Seite ein Schlüssel liegt (Konfiguration), wozu dieser zu verwenden ist und wann er zu wem übertragen wird.

Request	Der Transmitter signiert den Request mit seinem privaten Schlüssel (Private Key) und sendet seinen öffentlichen Schlüssel (Public Key) in seinem Request mit. Auf Seiten des Distributors wird der mitgeschickte öffentliche Schlüssel gegen das Swisssdec CA-Zertifikat geprüft und die Signatur verifiziert. Serverseitig wird also nur das Swisssdec CA-Zertifikat benötigt, um die Signatur zu prüfen. Der Transmitter verschlüsselt den Request mit dem öffentlichen Schlüssel des Distributors und nur der Distributor kann diese Daten dann mit seinem privaten Schlüssel entschlüsseln.
Response	Der Distributor signiert die Antwort mit seinem privaten Schlüssel. Dem Transmitter muss dessen öffentlicher Schlüssel, beziehungsweise das CA-Zertifikat bekannt sein. Der Distributor verschlüsselt den Response mit dem öffentlichen Schlüssel des Transmitters (hier mit dem UID-Zertifikat) und nur der Transmitter des entsprechenden Unternehmens kann diese Daten dann mit seinem dazugehörigen privaten Schlüssel entschlüsseln.

Tabelle 5.3. Bemerkungen zum Application Layer

A Referenzen

- [WSS] *Web Services Security*. SOAP Message Security 1.0 (WS-Security 2004). OASIS. 200401. <http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-soap-message-security-1.0.pdf> .
- [WSSX509] *Web Services Security*. UsernameToken Profile 1.0. OASIS. 200401. <http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-username-token-profile-1.0.pdf> .
- [RFC2246] *The TLS Protocol*. IETF. Version 1.0, 1999. <http://www.ietf.org/rfc/rfc2246.txt> .
- [ACKNOTIF] *Richtlinien für Lohndatenübermittlung*. Quittierung und Notifications. Swissdec. *AcknowledgmentNotification_d.pdf*. <https://infopoint.swissdec.ch/datapool.xhtml> .
- [DETSPEZ] *sUA_DetailSpezifikation_V1.5_10.05.2019*. Swissdec und BFH. Version 1.5, Ausgabe 23.05.2019.